Week 3, Day 1: Microservice Principles, API Gateway, Discovery, Config Server and LRU Cache

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: microservice principles, decomposition, API Gateway, service discovery, config server | DSA: Linked lists

Today's Schedule

- **0:00–0:50** Study microservice principles, decomposition and the trade-offs (sections 1 and 2).
- **0:50–1:20** Study the API Gateway (section 3).
- **1:20–1:50** Study service discovery and load balancing (section 4).
- **1:50–2:10** Study the config server and externalized configuration (section 5).
- **2:10–3:10** DSA: solve the 3 linked list problems (section 7). Spend at most 25 minutes per problem before reading the hint.
- **3:10–3:40** Answer the interview questions aloud (section 8) and add 10 flashcards.
- **Optional (60 min):** build the skeleton in section 6 so Week 3, Day 6 starts faster.

1. Microservice Principles

What a microservice is: a small, independently deployable service built around a **business capability** (orders, payments, notifications), with its own codebase, its own data and its own release cycle. Services talk to each other through APIs or events.

Core principles

- **Single business capability:** a service does one thing well and owns it end to end.
- **Independent deployability:** you can release one service without redeploying the others. If every release needs all services deployed together, you do not have microservices.
- **Database per service:** each service owns its data, and other services access it only through its API or events. A shared database creates hidden coupling.
- **Smart endpoints, simple pipes:** business logic lives in services, and communication is plain HTTP or messaging.

- **Design for failure:** networks fail, services are slow, instances restart. Timeouts, retries and circuit breakers (Day 3) are normal parts of the design.
- **Decentralized ownership:** a team owns a service from code to production ("you build it, you run it").
- **Automation:** CI/CD, containers, infrastructure as code and monitoring are required, not optional.
- **Twelve-factor ideas:** configuration from the environment, stateless processes, logs written to standard output, fast startup and graceful shutdown.

2. Decomposition, Trade-offs and When Not to Use Microservices

Benefits

- Scale only the services that need it, and deploy them independently.
- Fault isolation: one failing service need not take down everything, if you design for it.
- Team autonomy and freedom to pick suitable technology per service.
- Smaller codebases that are easier to understand and replace.

Costs (be honest about these in interviews)

- **Distributed system complexity:** network latency, partial failures and timeouts.
- **Data consistency:** no single ACID transaction across services, so you need sagas and eventual consistency (Day 4).
- **Operational overhead:** many pipelines, deployments, configs, dashboards and alerts.
- **Harder testing and debugging:** a request touches many services, so you need distributed tracing and contract tests.
- **Cost:** more infrastructure and more people with the right skills.

How to split a system

- **By business capability:** what the business does (ordering, billing, inventory).
- **By bounded context (Domain-Driven Design):** each context has its own model and language. A customer in billing is not the same thing as a customer in support.
- **By data ownership:** if two parts constantly need the same tables, they probably belong together.
- **Avoid splitting by technical layer** (a UI service, a database service). That produces chatty, tightly coupled services.

- **Strangler fig pattern:** to migrate a monolith, put a facade in front, move one capability at a time into a new service, and route traffic gradually until the old code can be retired. This is safer than a big-bang rewrite.

Warning signs of a distributed monolith

- Services share a database or deploy in lockstep.
- Long chains of synchronous calls where one slow service stalls everything.
- A change to one service regularly forces changes in many others.

A modular monolith (one deployable, with strict module boundaries) is often the right starting point. Move to microservices when team size, scaling needs or release independence justify the cost. Good candidates to mention are a small team, an early-stage product, or a domain you do not understand yet.

3. API Gateway

What it does: a single entry point for clients that routes requests to the right service.

- **Routing** by path, host, header or method.
- **Cross-cutting concerns in one place:** authentication and token validation, rate limiting, CORS, TLS termination, logging, tracing headers, request size limits.
- **Shielding clients** from internal service layout, so you can move or split services without changing clients.
- **Optional extras:** response caching, request aggregation, canary and A/B routing, protocol translation.

Spring Cloud Gateway

- Built on Spring WebFlux and Netty (reactive, non-blocking). Concepts: **route** (id, target URI, predicates, filters), **predicate** (when does a route match, for example Path, Method, Header), **filter** (modify the request or response, for example StripPrefix, AddRequestHeader, RequestRateLimiter, CircuitBreaker).
- A route target of **lb://service-name** means look up this service in the registry and load balance across its instances.
- Property names have moved between releases, so check the documentation for the Spring Cloud version you use. The shape below is the classic one.

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=0
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 20
                redis-rate-limiter.burstCapacity: 40
        - id: inventory-service
          uri: lb://inventory-service
          predicates:
            - Path=/api/inventory/**
```

Design points

- **Validate the JWT at the gateway** (it can act as an OAuth2 resource server) and pass identity to services in a header or by forwarding the token. Services should still enforce their own authorization, because the gateway is not the only possible caller on an internal network.
- **Keep business logic out of the gateway.** It should route and apply cross-cutting policy only.
- **Avoid a single point of failure:** run several gateway instances behind a load balancer.
- **Backend for Frontend (BFF):** a separate gateway or aggregation layer per client type (web, mobile) when their needs differ a lot.
- **Alternatives:** Kong, NGINX, Envoy, AWS API Gateway, Azure API Management. In Kubernetes, an Ingress controller or Gateway API often covers the edge routing role.
- **Gateway vs load balancer:** a load balancer spreads traffic across identical instances, mostly at layer 4 or 7 with little application awareness. A gateway makes API-aware decisions (auth, per-route rate limits, transformations).

4. Service Discovery and Load Balancing

The problem: in the cloud and in containers, instances start, stop and move, so their addresses change. Hard-coding hosts and ports does not work.

How discovery works

1. When a service instance starts, it **registers** its name, host and port with a **service registry**.
2. It sends **heartbeats** to renew its registration. If heartbeats stop, the registry removes it after a timeout.
3. Clients **look up** instances by service name and call one of them.

Eureka (Spring Cloud Netflix)

- **Server:** a Spring Boot app annotated with @EnableEurekaServer.
- **Client:** add spring-cloud-starter-netflix-eureka-client and point it at the server with eureka.client.service-url.defaultZone. The application name (spring.application.name) becomes the service id.
- **Defaults to know:** clients renew every 30 seconds, an instance expires after about 90 seconds without renewal, and clients cache the registry locally and refresh it about every 30 seconds. So a dead instance can still receive calls for a short time, which is why retries and circuit breakers matter.
- **Self-preservation mode:** if many heartbeats are missed at once, Eureka assumes a network problem and stops evicting instances, to avoid wiping the registry by mistake. Expect stale entries during network trouble.
- Eureka is still supported through Spring Cloud, but many teams on Kubernetes use the platform's own discovery instead.

Client-side vs server-side discovery

- **Client-side:** the caller asks the registry and picks an instance itself (Eureka plus **Spring Cloud LoadBalancer**, which replaced Netflix Ribbon). Flexible, but every client needs the logic.
- **Server-side:** the caller sends the request to a stable address, and a load balancer or the platform chooses an instance. **Kubernetes Services and DNS** work this way, as do cloud load balancers. Simpler for clients.
- **Other registries:** Consul, ZooKeeper, etcd.

Calling a service by name

```
@Configuration
class WebClientConfig {
    @Bean
    @LoadBalanced
    WebClient.Builder loadBalancedWebClientBuilder() { return
WebClient.builder(); }
}

// inventory-service is resolved through the registry and load balanced
Mono<Stock> stock = builder.build().get()
    .uri("http://inventory-service/api/inventory/{sku}", sku)
    .retrieve().bodyToMono(Stock.class);
```

Load balancing strategies: round robin (the default in Spring Cloud LoadBalancer), random, weighted, and least connections in some tools. Health checks should remove unhealthy instances from rotation, so expose /actuator/health with meaningful readiness checks.

5. Config Server and Externalized Configuration

The problem: with dozens of services and several environments, configuration copied into each jar or repository becomes unmanageable, and changing a value means rebuilding and redeploying.

Spring Cloud Config

- **Server:** a Spring Boot app with @EnableConfigServer that serves configuration, typically from a **Git repository**. You get versioning, history, review and rollback for free.
- **Files are organized by application name and profile:** for example order-service.yml, order-service-prod.yml and an application.yml with shared defaults.
- **Client:** in Spring Boot 2.4 and later, import the server with **spring.config.import=optional:configserver:http://config-server:8888** in the client's application config (the old bootstrap.yml approach is no longer needed). Set spring.application.name and the active profile so the right files are selected.
- **Refreshing at runtime:** mark beans with **@RefreshScope** and call the /actuator/refresh endpoint to reload changed values without restarting. To refresh many instances at once, use **Spring Cloud Bus** over Kafka or RabbitMQ.
- **Secrets:** do not commit plain secrets to Git. Use encrypted values, or better, a secrets manager such as HashiCorp Vault, AWS Secrets Manager or Kubernetes Secrets.

- **Availability:** the config server is critical at startup. Run more than one instance, and configure retry and fail-fast behavior on clients.

```
# client application.yml
spring:
  application:
    name: order-service
  config:
    import: "optional:configserver:http://localhost:8888"
  profiles:
    active: dev
```

Alternatives, especially on Kubernetes or cloud

- Kubernetes **ConfigMaps and Secrets** mounted as files or environment variables.
- AWS Systems Manager Parameter Store and Secrets Manager, Consul KV, Azure App Configuration.
- Many teams skip a dedicated config server and rely on environment variables plus a secrets manager, which fits the twelve-factor approach.

Precedence reminder: command-line arguments and environment variables override values from files, and externally served config sits above packaged defaults.

6. Optional Hands-On: Build the Skeleton (60 Minutes)

This is the start of the Week 3 to 4 mini project, and doing it now makes Day 6 lighter.

1. Create four Spring Boot 3 projects: **discovery-server** (Eureka), **config-server** (with a local Git folder as the backend), **api-gateway** (Spring Cloud Gateway), and **order-service** (reusing your secured API from Week 2, Day 6 if you like).
2. Start in this order: config server, discovery server, order service, gateway. Check the Eureka dashboard on port 8761 and confirm that the order service is registered.
3. Add a second route for a tiny **inventory-service** and call both through the gateway.
4. Run two instances of the order service on different ports and watch the gateway round robin between them.
5. Stop one instance and observe how long calls keep going to it, then relate that to the heartbeat and cache timings above.
6. Commit it all to GitHub with a README that has a simple architecture sketch.

You will be able to explain from experience why retries and circuit breakers exist, which is a much stronger interview answer than the definition.

7. DSA: Linked Lists (3 Problems)

These use the same ListNode class as Week 2, Day 4.

Problem 1: Remove Nth Node From End of List

- **Idea:** one pass with two pointers. Use a **dummy head**. Move the fast pointer n steps ahead, then move both until fast reaches the last node. The slow pointer is then just before the node to remove.
- **Complexity:** O(n) time, O(1) space.
- **Edge cases:** removing the head (the dummy node handles it), a single-node list, n equal to the list length.

```
public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next != null) {
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;
    return dummy.next;
}
```

Problem 2: Reorder List

- **Idea:** three known steps. Find the middle with slow and fast pointers, reverse the second half, then merge the two halves alternately (first, last, second, second last, and so on).
- **Complexity:** O(n) time, O(1) space.
- **Edge cases:** one or two nodes, odd and even lengths. For 1,2,3,4 the result is 1,4,2,3, and for 1,2,3 it is 1,3,2.

```
public void reorderList(ListNode head) {
    if (head == null || head.next == null) return;
    ListNode slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    ListNode second = reverse(slow.next);
    slow.next = null;                       // cut the list in two
    ListNode first = head;
    while (second != null) {
        ListNode t1 = first.next, t2 = second.next;
        first.next = second;
        second.next = t1;
        first = t1;
        second = t2;
    }
}

private ListNode reverse(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Problem 3: LRU Cache

- **Idea:** combine a **HashMap** (key to node, for O(1) lookup) with a **doubly linked list** ordered by recency. On every get or put, move the node to the front. When the cache is full, evict the node at the back. **Sentinel head and tail nodes** remove the null checks.
- **Complexity:** O(1) for both get and put, O(capacity) space.
- **Edge cases:** updating an existing key (do not evict), capacity of 1, a get on a missing key.
- **Thread safety:** this version is not thread-safe. In production use a library such as Caffeine, or synchronize carefully.

```
class LRUCache {
    private static class Node {
        int key, val;
        Node prev, next;
        Node(int k, int v) { key = k; val = v; }
    }

    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0), tail = new Node(0, 0);

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        Node n = map.get(key);
        if (n == null) return -1;
        remove(n);
        addFront(n);
        return n.val;
    }

    public void put(int key, int value) {
        Node n = map.get(key);
        if (n != null) {
            n.val = value;
            remove(n);
            addFront(n);
            return;
        }
        if (map.size() == capacity) {
            Node lru = tail.prev;
            remove(lru);
            map.remove(lru.key);
        }
        n = new Node(key, value);
        map.put(key, n);
        addFront(n);
    }
```

```
    private void remove(Node n) { n.prev.next = n.next; n.next.prev = n.prev;
}
    private void addFront(Node n) {
        n.next = head.next;
        n.prev = head;
        head.next.prev = n;
        head.next = n;
    }
}
```

Shortcut worth knowing (Day 3 callback): LinkedHashMap with accessOrder set to true plus an overridden removeEldestEntry gives you an LRU cache in a few lines. Interviewers usually want the manual version first, then ask about the shortcut.

```
class SimpleLru<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    SimpleLru(int capacity) { super(16, 0.75f, true); this.capacity =
capacity; }
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}
```

8. Interview Questions to Answer Aloud

1. What is a microservice, and how is it different from a monolith? What does independent deployability mean?
2. What are the benefits and the costs of microservices? When would you not use them?
3. How do you decide service boundaries? What is a bounded context?
4. What is the strangler fig pattern?
5. Why should each service own its database, and what problems does that create?
6. What is a distributed monolith, and how do you spot one?
7. What does an API Gateway do? What should it not do?
8. How is an API gateway different from a load balancer?
9. How does Spring Cloud Gateway route a request to lb://order-service?
10. How does service discovery work? Explain registration, heartbeats and lease expiry in Eureka.
11. Client-side vs server-side discovery: how do Eureka and Kubernetes differ?

12. How would you manage configuration across 20 services and 4 environments? How do you handle secrets?
13. How do you refresh configuration without restarting, and what are the risks?

9. End-of-Day Checklist

- [ ] I can state the principles of microservices and explain the costs honestly.
- [ ] I can describe how to split a system and what a distributed monolith looks like.
- [ ] I can explain what a gateway does, and write a route with predicates and filters.
- [ ] I can explain registration, heartbeats, client-side caching and why stale entries happen.
- [ ] I can explain a config server, @RefreshScope, and the alternatives on Kubernetes.
- [ ] I solved all 3 linked list problems and can explain the LRU cache design.
- [ ] (Optional) I built the skeleton and saw two instances being load balanced.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 3, Day 2): inter-service communication (REST, Feign, gRPC basics), synchronous vs asynchronous calls and timeouts, plus queue and sliding window problems.